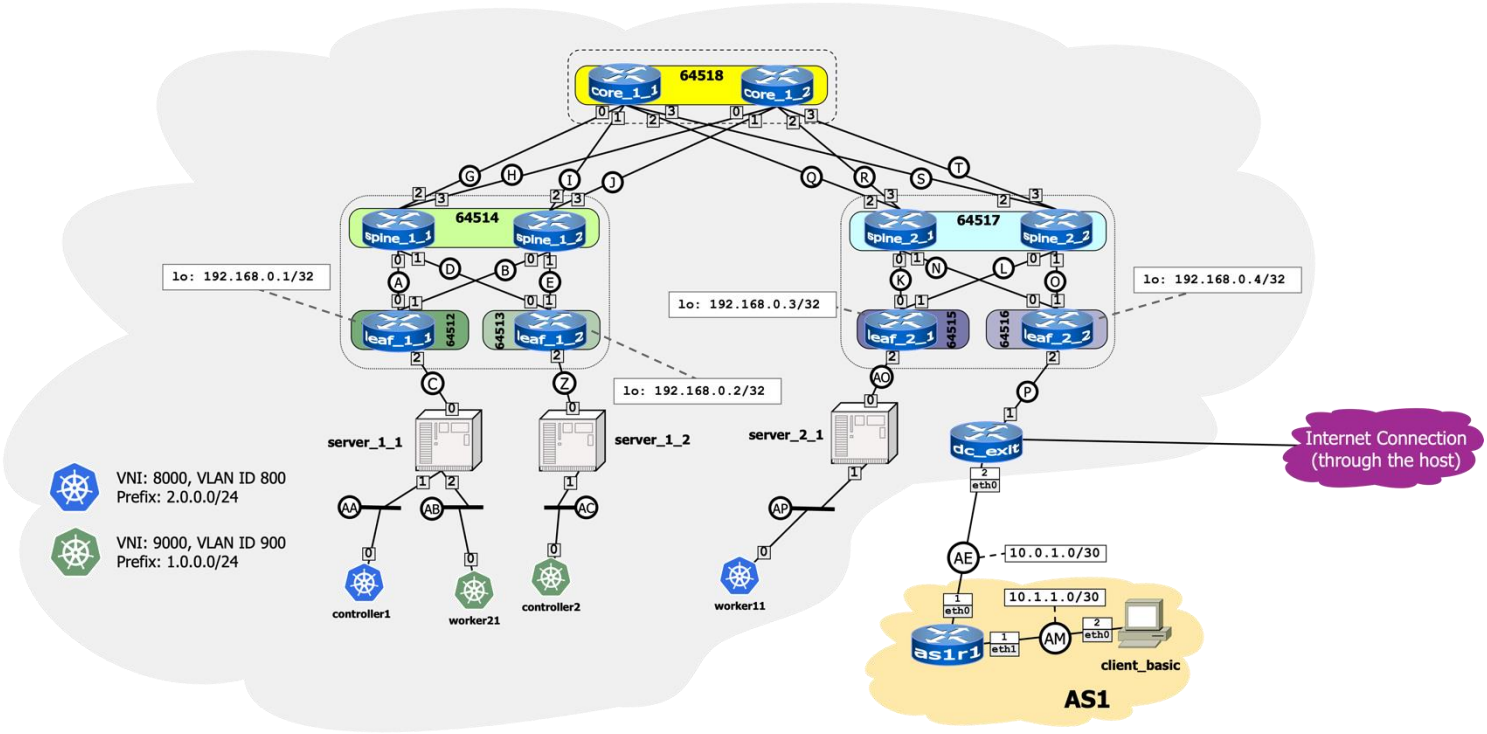
BASIC

In the basic project, you will build a Cloud infrastructure that offers virtualized environments where tenants can deploy their own Kubernetes clusters. The figure below illustrates the reference design for the Cloud infrastructure (gray cloud) including its interconnection with an external access network called AS1. You decided to rely on BGP as the underlay routing protocol. You managed to sign two contracts with your first two customers who will use your infrastructure to deploy their services. The two customers — called Tenant A (blue) and Tenant B (green) from now on — require separate Kubernetes clusters — Cluster A (blue VNI) and Cluster B (green VNI) — which will run as virtual machines (VMs) on bare metal servers. To guarantee complete isolation between the two clusters, each cluster must be operated on its own VXLAN-based overlay network. Additionally, to enable efficient VXLAN communication, the cloud provider must enable and configure EVPN BGP within the Fat-Tree network.



***Note**: Kubernetes worker nodes, `dc_exit`, and `as1r1` are already configured and provided in the lab.*

Part 1: Network Configuration

Using Kathará, implement the network depicted above and described below:

- ☐ Create the Fat-Tree topology (routers and servers)
  - The grey boxes at the endpoint of a link are the interface numbers (the prefix `eth` is often omitted for brevity)
    - For example, `server_1_1` has three interfaces labelled `0`, `1`, and `2`, corresponding to `eth0`, `eth1`, and `eth2`.
  - The white box above the interface number is the last byte of the IP address in the subnet
    - For example, if the subnet is `10.1.1.0/30`, and the white box number is 2, the IP is `10.1.1.2/30`
    - If the white box is omitted, no IP address needs to be configured on the interface
  - You need to specify all the devices metadata (**hint:** take look at the `lab.conf` file of Lab 2)
- ☐ Configure BGP (both underlay and EVPN)
  - Select a unique Router ID of your choice for each router
  - Use unnumbered peerings with link-local IPv6 addresses (remember to enable IPv6 in `lab.conf`)
  - On Leaf nodes, configure the peerings on the loopback port (IP of each Leaf shown)
  - Advertise correct address families to neighbours (both IPv4 unicast and EVPN)
    - **Hint:** remember to add the `route-map` for announcing the loopback IP
- ☐ Configure VXLAN and VLAN on the Leaf nodes and the servers
  - You must configure **ONLY** the required VTEPs/VLANs in each node

        - E.g., `server_1_2` must only have the VTEP for VNI=9000 and VLAN=900
    - The only exception is `leaf_2_2`, that must expose both VTEPs/VLANs
        - They allow the Kubernetes nodes to exit on the host's Internet connection
- ☐ After configuring the network, check that the two Kubernetes clusters are healthy
    - Log into `controller1` and `controller2` separately and check that all system resources are running
- ☐ Configure `client_basic` interface and its default gateway (towards `as1r1`)
    - Its configuration folder with all the required files is already provided by the lab

Part 2: Microservices Deployment

Both tenants are ready to deploy microservices in their respective clusters.
End users can access Cluster A via the `clustera.com` domain and Cluster B via the `clusterb.com` domain.
Both Tenant A and Tenant B plan to offer a web-based service that utilizes a compact storyteller LLM (LLama2-15M) to generate two types of stories:

1. **onceupon (deployed by Tenant A in Cluster A)**: Produces narratives that begin with "Once upon a time…"
2. **faraway (deployed by Tenant B in Cluster B)**: Produces narratives that begin with "Long ago, in a faraway land…"

The cloud provider grants both clusters access to a shared storage system where the LLM weights are stored, using a Kubernetes PersistentVolume (PV) exposed through the storage class named "weights".

Define the resources needed to deploy the two microservices in the two clusters. You can write the definitions in the `shared/` folder of the lab on your host. Here, you will find two folders, `clustera` and `clusterb`, that you can use to organize the different resources. The specifications of the microservices are the following:

- ☐ All the resources MUST be enclosed into a Kubernetes namespace called `llm-ns`
- ☐ The microservice should be exposed through a Service resource with **port 8000**
- ☐ The weights should be accessed by Pods through the PV `weights` with 100MB of claim
- ☐ The microservice should have only one replica, you must ensure the rescheduling of the Pod in case of failures
    - The Pod container must expose **port 8080** (that needs to be connected to the Service port!)
    - The container image is `ik2227/llama2`
    - The container must define the following environment variable:
        - On Cluster A:
            - `name: LLAMA_SYS_PROMPT`
            - `value: "Once upon a time,"`
        - On Cluster B:
            - `name: LLAMA_SYS_PROMPT`
            - `value: "Long ago, in a faraway land,"`
    - The PVC must be mounted in the `/data` folder of the container
- ☐ On Cluster A, the microservice should be reachable:
    - By accessing the corresponding domain (`clustera.com`)
    - By hitting the `/completion` path
- ☐ On Cluster B, the microservice should be reachable:
    - By accessing the corresponding domain (`clusterb.com`)
    - By hitting `/completion` path

Part 3: Testing

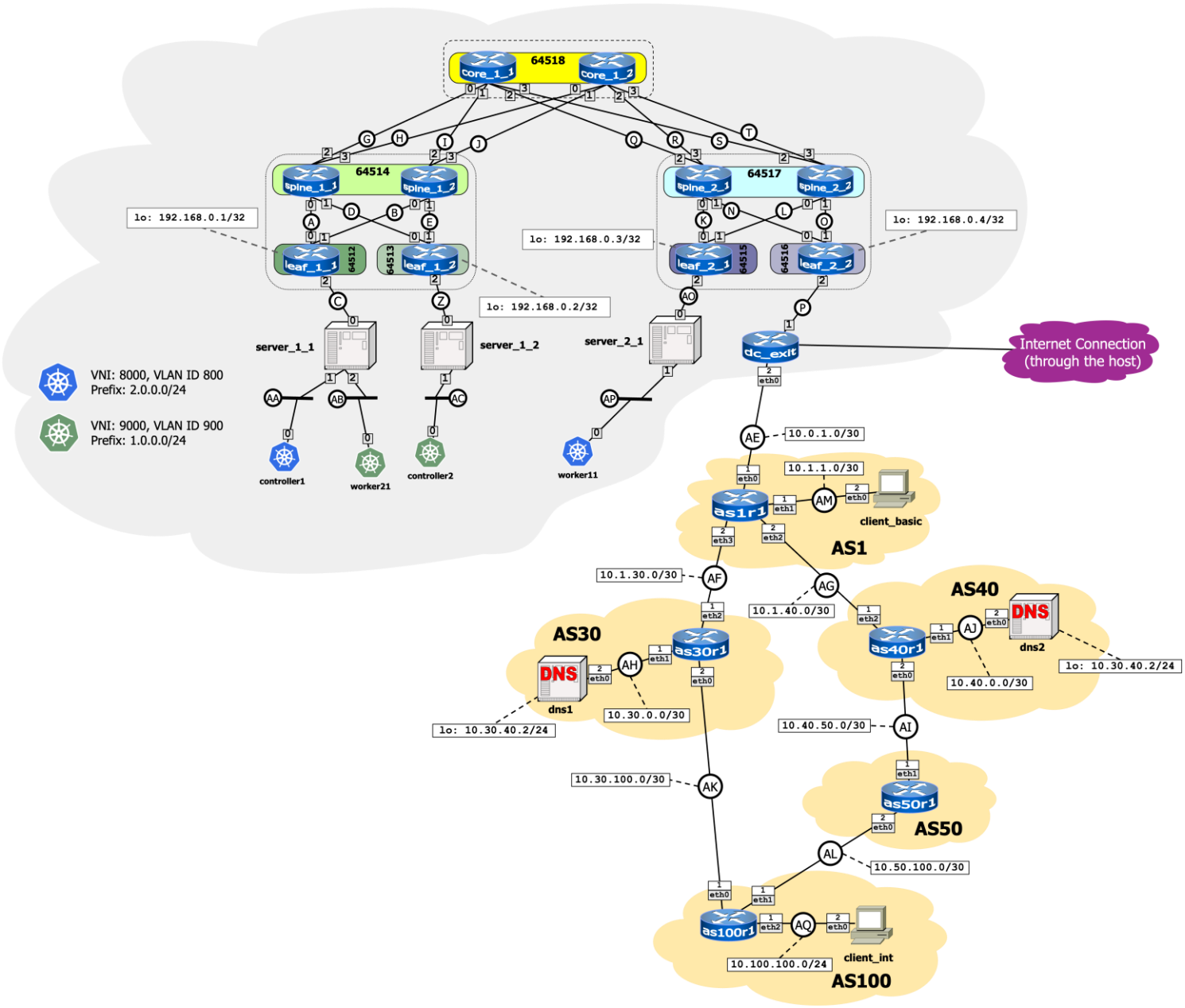
- ☐ After deploying the microservices, check that both are reachable from `client_basic`:
    - You can use the following command to craft a correct request:

```
curl -X POST -H "Content-Type: application/json" -d '{"query": "An orc wanted to destroy everything"}' http://<cluster_domain>/completion
```

    - You can change the query text with anything you like! ☺
    - The microservice should return a JSON with few sentences generated by the LLM and starting with the right introduction (either "Once upon a time" or "Long ago, in a faraway land").

INTERMEDIATE

As the cloud provider's network expands, it begins adopting anycast policies to enhance end-user access to its services. Specifically, the provider deploys two anycast DNS servers across different Autonomous Systems (AS30 and AS40, respectively). Both servers store identical DNS records for accessing Cluster A and Cluster B, ensuring high availability if one DNS fails.



Part 1: Network Configuration

Using Kathará, expand the network obtained in the Basic section with the new devices shown above and described below:

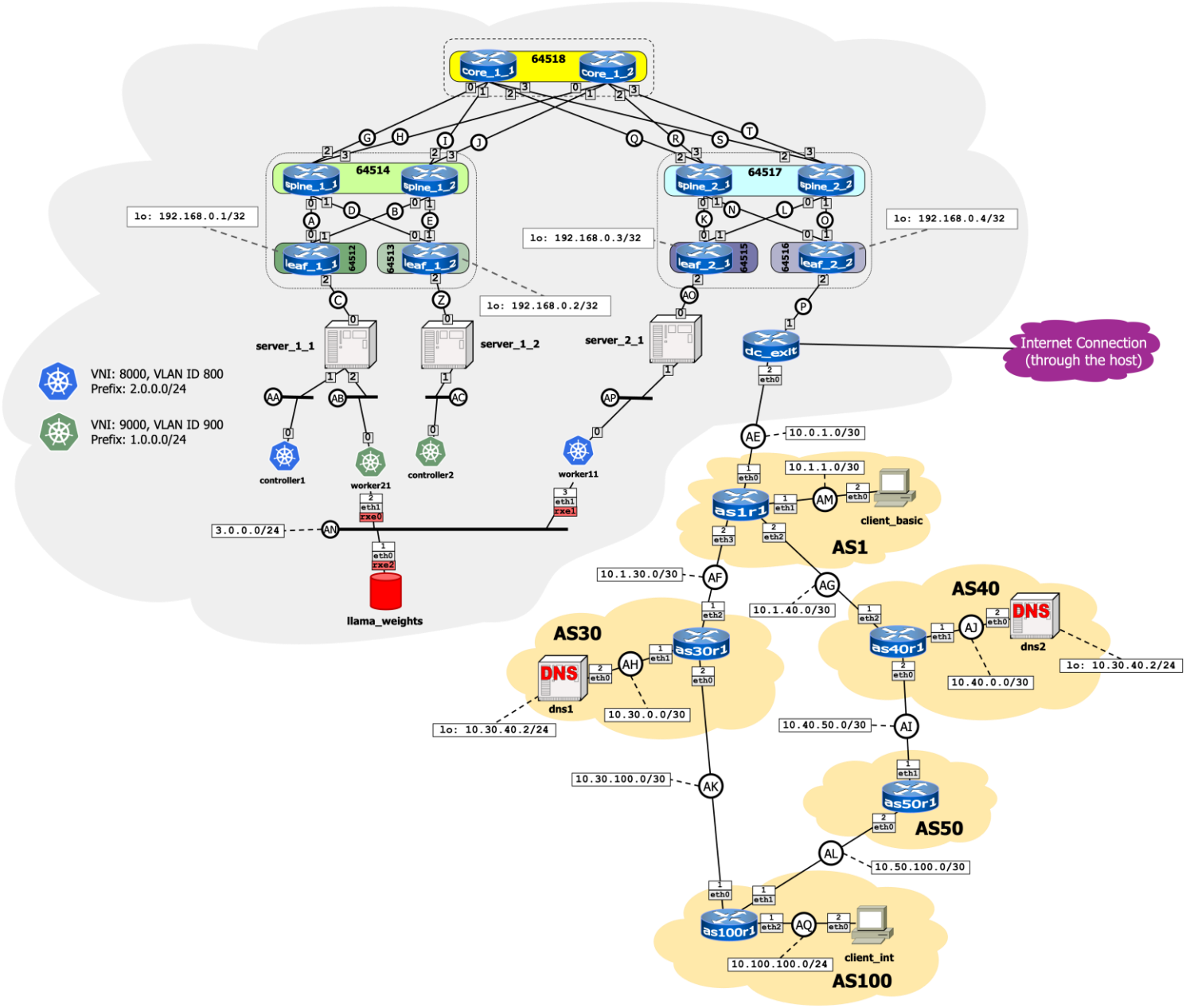
- ☐ Modify `as1r1` with two additional interfaces towards AS30 and AS40, respectively
  - Configure the eBGP peerings towards AS30 and AS40
- ☐ Create routers in AS30, AS40, AS50 and AS100 and configure eBGP among them
  - AS100 announces the **10.100.100.0/24 network**
- ☐ Create and configure `dns1` and `dns2`
  - Both are root NS and have authority on two A records:
    - `clustera.com.` with **IP 10.0.200.1**
    - `clusterb.com.` with IP **10.0.100.1**
  - Both DNS have an iBGP peering with their nearest router
  - Both DNS announce on BGP the IP subnet associated with the loopback interface (shown in the figure)
  - Both DNS run a health-check script that announces/withdraws the loopback IP subnet based on the status of BIND
- ☐ Configure `client_int` interface and its default gateway (towards `as100r1`)
  - Its configuration folder with all the required files is already provided by the lab

Part 2: Testing

- ☐ Verify which of the two DNS is answering to the client
- ☐ Verify that the DNS is correctly resolving the hostnames `clustera.com` and `clusterb.com`
- ☐ Verify that, breaking the connection towards the main DNS, anycast works correctly (the backup DNS should answer)
- ☐ Verify that, by shutting down the BIND service, the backup DNS answers correctly
- ☐ Check that both microservices are reachable from `client_int` (using the same cURL command of Basic)

ADVANCED

Both Tenant A and Tenant B have identified an inefficiency in their microservice: the LLama2 library reloads the weights from a file every time a user submits a new query. However, the cloud provider offers a backend storage network containing the LLM weights that is RDMA-enabled. Specifically, the `llama_weights` device loads Llama2's weights into its local memory and operates as an RDMA-enabled server, listening for clients and establishing an Infiniband connection with them. Consequently, the microservices can be optimized to fetch the weights via RDMA from `llama_weights` and load them into local memory only at application startup.



***Note**: `llama_weights` is already configured and provided in the lab.*

Part 1: Network Configuration

Using Kathará, expand the network obtained in the Intermediate section as described below:

- ☐ Modify `worker11` and `worker21` by adding the additional `eth1` interface
  - Configure the IP address of `eth1`
- ☐ Create the SoftRoCE interfaces on `worker11` and `worker21` with the names shown in the figure

Part 2: Microservices Configuration

- ☐ Create a new YAML file for the Deployment definition, copying the one from the Basic section
- ☐ The Deployment needs some additions to work in "RDMA mode"
  - After the image specification, add the following snippet to give full privileges to the container:
    ```
    securityContext:
      privileged: true
    ```
  - Add the following environment variables after the `LLAMA_SYS_PROMPT`:
    - `name: ENABLE_RDMA`
    - `value: "true"`
    - `name: RXE_IFACE`
    - `value: <name_of_softroce_iface_of_the_worker>`
      - For example, `rxe1` when deploying on Cluster A
    - The two variables will change the way the webservice code behaves
  - We also need to change the volume mounts of the containers:
    - Remove the volume mount to the `/data` folder (associated with the PVC)
    - Add the following volume mounts:
      ```
      spec:
        containers:
        - ...
          volumeMounts:
            - mountPath: /data/roce
              name: roce-client
            - mountPath: /dev/infiniband
              name: ib
        volumes:
        - name: roce-client
          hostPath:
            path: /shared/roce
        - name: ib
          hostPath:
            path: /dev/infiniband
      ```
    - `roce-client` will mount the `shared/roce` folder from your host lab directory
    - `ib` will give the container visibility to the SoftRoCE interface
  - Since we need RDMA support, we must instruct the Kubernetes scheduler to deploy the Pods only in RDMA-enabled nodes (`worker11` and `worker21`). To do so, the workers have a specific label `supports=rdma` to help the Kubernetes scheduler. You can specify the node selector as follows:
    - After the `volumes` specification, add the red snippet:
      ```
      spec:
        containers:
        - ...
        volumes:
        - ...
        nodeSelector:
          supports: rdma
      ```
  - **Do not apply the Deployment! The RDMA client code is still missing!**

Part 3: Writing RDMA Client Code

- ☐ The `shared/roce` folder from your host lab directory contains a file `rdma.py`
  - This file is loaded by the microservice when working in "RDMA mode" and should implement the RDMA client logic to remotely read the weights from the `llama_weightsr`evice (that works in RDMA server mode)
  - The file is mounted in the Pod's containers through the `roce-client` volume
    - Every modification is mirrored into the microservice
- ☐ The file contains some constants and functions
  - Importantly, the `BUFFER_SIZE` constant specifies the size of the local memory, exactly sized to fit the LLama2 weights
  - The `poll_cq` will wait for a specific WR ID to stop and return the weights to the microservice
- ☐ Your task is to write the logic into the `read_weights` function that:
  - Create an Infiniband connection with `llama_weights` acting as a **client**
  - Starts the CQ polling thread to process completion events (CQEs)
  - Posts a RDMA READ request (`opcode` is `IBV_WR_RDMA_READ`)
    - The SGE needs to be prepared to write the received data from the beginning of the local memory area, the length value should be equal to the size of the LLama2 weights (see the constants description above)
    - The WR **MUST** have WR ID equal to `0xdead`
    - The WR specifies to the read operation to fetch from the first location of the remote memory area

Part 4: Testing

- ☐ Apply the new Deployment
- ☐ Verify that the RDMA connection is working (put some logging into your code)
  - From client side, you can use `kubectl -n llm-ns logs <pod_name>` to check the webserver logs
  - From server side, you can connect to the `llama_weights` device and check the RDMA server logs:
    - `tail -f /var/log/server.log`
- ☐ If no errors are shown, check that both microservices are reachable from `client_int` (using the same cURL command of Basic) and correctly return a generated response